Pixels-to-Predictions: SmolVLM-500M QLoRA Fine-Tuning for ScienceQA

Abstract

We fine-tune the open-source vision-language model `HuggingFaceTB/SmolVLM-500M-Instruct` for the *Pixels-to-Predictions* multimodal multiple-choice science QA competition under a strict 5,000,000-trainable-parameter budget and an offline-evaluation rule. We apply QLoRA — 4-bit NF4 quantization with double quantization on the base model and rank-8 LoRA adapters across all seven linear projections in every text-decoder layer — yielding 4,341,760 trainable parameters (87% of the cap). We score predictions via single-token log-likelihood at the answer-letter position and apply a position-bias-aware test-time augmentation (smart-TTA) that averages logits across $K = 4$ random choice permutations while bypassing TTA on 2-choice questions where it empirically degrades accuracy. Our best configuration ($r = 8$, all-projection LoRA, 384 px images, 5 training epochs) achieves a public leaderboard score of **0.81287**, a +4.83 percentage point improvement over the same model with standard scoring (0.76458). For reference, the same architecture with no fine-tuning scored ~ 41.5% on a 200-example val proxy — though this small-sample estimate (binomial SE $\approx$ 3.5 pp) is not directly comparable to the 1,008-example test split, the qualitative gap from low-40s to low-80s confirms that fine-tuning is the dominant source of accuracy. Five ablations isolate which axes of capacity matter at this scale: training duration is the dominant lever (+3.6 pp LB from 3 to 5 epochs), but a further extension to 7 epochs *reduces* LB by 1.0 pp (overfitting), establishing 5 epochs as near-optimal; doubling LoRA rank from 4 to 8 contributes +1.8 pp at matched training duration; higher visual resolution (384 → 512 px) is statistically tied with baseline.

1 Introduction

The competition task is multimodal multiple-choice science question answering. Each example pairs an image (diagrams, maps, charts, photos) with a natural-language question, 2–5 textual answer choices, and optional pedagogical context (`hint`, `lecture`). The dataset matches the ScienceQA schema.

Hard constraints. Submissions must use only `HuggingFaceTB/SmolVLM-500M-Instruct` as the base model; the submission notebook must run with internet OFF; compute is restricted to free-tier Kaggle (T4); trainable parameters must not exceed 5,000,000. These rules eliminate full fine-tuning of the 511M-parameter base, late-stage `pip install` of inference-time dependencies, and ensembling with non-SmolVLM models. They also make compute economy first-order: a single 3-epoch full-data run on T4 takes ~ 2.7 hours; the 30 GPU-hour weekly quota supports roughly 8 such runs.

Random and pretrained baselines. The dataset's `num_choices` distribution is approximately 21 / 50 / 25 / 4% over $\{2, 3, 4, 5\}$, giving a random-guess baseline of ~ 32.5%, *not* 25%. The base SmolVLM-500M, used zero-shot via single-token log-likelihood scoring, achieves **41.5%** on a 200-example val sample (binomial SE $\approx$ 3.5 pp). This baseline exhibits a strong "A" bias: 71.5% of predictions choose the first listed option, and 5-choice questions underperform random (18.6%) — we hypothesize this is because the letter "E" rarely appears as a multiple-choice option in pretraining text, suppressing its log-prior. These two patterns motivate the loss-masked SFT and choice-permutation TTA design choices below.

Contributions.

1. A QLoRA fine-tuning pipeline targeting all seven text-decoder linear projections at rank 8, sized exactly to 87% of the 5M trainable-param budget.
2. A position-bias-aware smart-TTA at inference that averages over $K = 4$ random choice permutations on 3+ choice questions while bypassing TTA on 2-choice questions where it empirically degrades accuracy.
3. A **data-curve-driven** (rather than random-search) ablation methodology: each ablation tested a hypothesis derived from the previous

run's loss curve or val/LB pattern, leading to five controlled experiments that isolate training duration, LoRA rank, image resolution, and TTA strategy — with the headline finding that training duration peaks at 5 epochs and overfits at 7.

4. Diagnosis and resolution of a QLoRA loss-explosion failure mode on T4: standard QLoRA stability practices (paged AdamW 8-bit, aggressive grad clipping at 0.3) plus an extended warmup ratio (0.1 vs. the typical 0.03), without which our first run diverged catastrophically at the first optimizer step.

Code: https://github.com/Palomarr/pixels-to-predictions.

2 Methodology

2.1 Architecture and Parameter Budget

`SmolVLM-500M-Instruct` consists of a SigLIP-Base vision encoder (~ 93M params, frozen), a small modality projector (frozen), and a SmolLM2-360M text decoder (32 transformer layers, hidden size 960, 15 attention heads, 5 KV heads, MLP intermediate 2,560). We adapt only the text decoder via LoRA — the budget can support adapting only one of the three components, and the text decoder is where multimodal reasoning chains form.

We adapt all seven linear projections in every transformer layer: four self-attention projections (`q_proj`, `k_proj`, `v_proj`, `o_proj`) and three Llama-style MLP projections (`gate_proj`, `up_proj`, `down_proj`). At rank $r = 8$, $\alpha = 16$ ($\alpha/r = 2$ heuristic), this produces **4,341,760 trainable parameters** (87% of the 5M cap). Three alternatives were considered (Table 1).

Allocation	r	**Trainable**	**Notes**
Attention-only (q, k, v, o)	16	3.28M	Wastes MLP capacity
Attention-only (q, v)	32	3.28M	Even narrower
MLP-only (g, u, d)	8	2.70M	Misses attention adaptation
All seven, $r = 8$	**8**	**4.34M**	**Selected**

Table 1: Considered LoRA allocations under the 5M cap. The QLoRA paper finds that low-rank adaptation across all linear layers beats higher-rank attention-only at matched budgets.

Module-targeting via regex. Passing `target_modules=[q_proj, …]` directly to PEFT would also match the *vision encoder's* attention projections, which share names with the text decoder's. This silently spends ~ 30% of the budget on adapting frozen vision features. We instead use a regex `target_modules` constrained to `model.text_model.layers.\d+\.(self_attn|mlp)\.…`, restricting adaptation to text-decoder layers only. The final module count is verified at training start: 224 modules matched, 4.34M trainable confirmed.

2.2 Quantization: QLoRA

We load the base model in 4-bit NF4 with double quantization. Compute dtype is technically `bf16` — PyTorch 2.10′s `is_bf16_supported()` returns True on T4 — but T4 is a Turing-architecture (sm_75) GPU whose tensor cores natively support fp16, not bf16; the ostensibly-bf16 path executes via emulated fallback kernels rather than dedicated tensor-core ops. We did not measure a fp16 alternative, so any throughput penalty is bundled into our wall-clock numbers in Section 3. Quantization (a) reduces base-model VRAM from ~ 1.0 GB (bf16) to ~ 250 MB (NF4), giving headroom for larger effective batches and longer prompts, and (b) keeps gradient flow well-behaved through dequantize-on-the-fly paths.

A practical complication arose at submission time: Kaggle's offline-evaluation environment lacks `bitsandbytes`, and the rule against internet access prevents `pip install` at test time. Infer-

ence therefore loads the base in `bf16` rather than 4-bit. The LoRA adapter — optimized assuming 4-bit-dequantized weights during training — is then applied to the original `bf16` weights. Empirically this introduces a ~ 3.4 pp gap between training-time val accuracy (4-bit) and inference-time val accuracy (bf16) on our v1 baseline (0.7634 vs 0.7290). We discuss this trade-off in Section 6.1.

2.3 Prompt Format and Loss Masking

We follow the competition starter notebook's prompt template to ensure the model sees the same structure at training and inference time:

```
<image>
Context:
{lecture, truncated to 800 chars}
{hint}

Question: {question}
Choices:
  A. {choice_0}
  B. {choice_1}
  ...
Answer: {letter — only at training}
```

The `<image>` token marks where the processor splices in visual tokens. We force `do_image_splitting=False` and `longest_edge=384`, producing exactly 81 visual tokens per image. The processor's default behavior is multi-tile splitting up to 1,536 px (~ 1,088 visual tokens), which would (a) blow past our 1,024-token sequence limit and trigger truncation that desyncs the visual-token block, and (b) waste compute on most ScienceQA images, which are $<$ 512 px natively.

Loss masking. During SFT we set cross-entropy `labels` to -100 (ignore) at every token position *except* the trailing answer-letter token. This concentrates 100% of the gradient signal on "predict the right letter" rather than wasting ~ 99% on modeling the lecture/question/hint text. Without this masking, our 4.34M-parameter budget would be largely consumed by language-modeling pressure on context tokens that have no test-time analogue. Loss masking cleanly aligns training and inference objectives.

2.4 Inference: Log-Likelihood Scoring

Rather than generating an answer free-form via `model.generate()` and parsing the output, for each choice index $j \in \{0, ..., \text{num_choices} - 1\}$ we compute

$$\text{score}[j] = \log P\big(\text{letter}_j \mid \text{image}, \text{prompt}\big)$$

where letter_j is the single token corresponding to the j-th letter (`" A"`, `" B"`, …). The argmax is the predicted answer index. This requires **one forward pass per example** — no autoregressive decoding — and is fully deterministic.

The leading space matters: `" A"` and `"A"` are different tokens under the SmolLM2 tokenizer. We pre-compute all letter-token IDs once and gather the next-token logits at the *last non-pad position* of each batch row (since right-padding shifts the answer position differently per example). Letters beyond `num_choices` are masked to $-\infty$ before argmax, so a 3-choice question can never predict "D".

This avoids the brittleness of generation parsing (variable formats like `(A)`, `A.`, `Answer is A`, hallucinated letters) and gives graceful behavior across all `num_choices` $\in \{2, ..., 5\}$.

2.5 Test-Time Augmentation: Smart Choice-Permutation

The pretrained-baseline analysis (see Section 4) revealed strong position bias: the model prefers `A` independent of choice content. After fine-tuning, residual position bias remains and is empirically larger when more letter positions are involved.

Choice-permutation TTA. For each test example, we run $K = 4$ forward passes. Pass 0 uses the original choice ordering (the identity permutation), so $K = 1$ reduces exactly to the no-TTA scoring path. Passes 1–3 each shuffle the choices into a random ordering, build the corresponding prompt, and score. To average logits in *original-choice space*, we map each pass's letter-position logit back to the original choice it represents:

```
accum[i, original_choice] +=
letter_logits[i, shuffled_position]
```

After K passes, $\arg\max_i$ over `accum[i]` selects the original choice with the highest *averaged* log-probability across orderings. Position bias cancels because each original choice now appears in every position roughly evenly.

Smart TTA: skip 2-choice. On val, $K = 4$ TTA hurts 2-choice questions by -1.23 pp while helping 3-, 4-, 5-choice by $+1.18$, $+2.78$, $+4.55$ pp respectively. With only two unique permutations available for 2-choice questions, $K = 4$ random sampling adds variance without canceling position bias (which is small at $n = 2$ anyway). We introduced a `tta_skip_below=3` parameter that bypasses TTA on examples with fewer choices than the threshold, falling back to the identity-pass prediction. Smart-TTA yields $+1.91$ pp val accuracy and $+1.21$ pp leaderboard accuracy on the v1 baseline — both larger than naive always-TTA.

2.6 Training Stability Fixes

Our first training run failed catastrophically: at step 20 the loss spiked to 2.66×10^{16}, and from steps 40–100 reported `0.000000` (gradient underflow). The pattern — stable forward+backward for the first 16 steps (one full gradient-accumulation cycle), then a single optimizer update destroying the model — pointed to three independent fp16 stability issues that compound under QLoRA.

Fix	What it addresses
`optim="paged_adamw_8bit"`	Default `adamw_torch` keeps optimizer states in fp32; under QLoRA + bnb mixed-precision dequant the scale mismatches amplify. The 8-bit paged optimizer is the reference QLoRA recipe.
`max_grad_norm=0.3` (was 1.0)	Default 1.0 is calibrated for full fine-tuning. With only 4.34M params receiving gradients flowing through 511M frozen-quantized params, per-parameter step sizes are effectively much higher; aggressive clipping is the standard QLoRA practice.
`warmup_ratio=0.1` (was 0.03)	At 0.03, only ~ 12 warmup steps preceded peak LR=2e-4; the first optimizer update fired at step 16 with LR essentially at peak. 0.1 gives ~ 58 warmup steps, so by step 16 the effective LR is ~ 40% of peak.

Table 2: QLoRA training-stability fixes that turned a divergent first run into reproducible training.

After applying all three fixes, training was stable from step 1 across all subsequent runs. We also added a defensive `torch.cuda.empty_cache() + synchronize()` before `set_seed()` to recover from corrupted CUDA state on Kaggle kernels that had inherited NaN/inf weights from prior failed runs.

3 Experimental Setup

Data splits. Train 3,109; val 1,048; test 1,008 (labels held out). Train is small relative to the model — at effective batch 16 and 3 epochs, only ~ 580 optimizer steps.

Optimizer & schedule. Paged AdamW 8-bit, learning rate 2×10^{-4}, cosine decay with 10% warmup, weight decay 0.0, max grad norm 0.3. Effective batch size 16 (per-device 1 $\times$ gradient accumulation 16). Mixed precision: nominal `bf16` per PyTorch 2.10′s reported support, though as noted in Section 6.1 the T4 (sm_75) lacks native bf16 tensor cores and these ops fall back to emulated paths.

Sequence and image limits. `max_seq_length=1024`. Lectures pre-truncated to 800 characters with word-boundary split. Images forced to a single 384×384 tile, ~ 81 visual tokens.

Hardware. NVIDIA T4 (16 GB) on Kaggle Notebooks Free Tier. Each 3-epoch run takes ~ 2:44 wall-clock; 5-epoch run ~ 4:50; 7-epoch run ~ 6:30. We avoid Kaggle's P100 due to a known sm_60 incompatibility with PyTorch 2.10.0+cu128.

Reproducibility. Single random seed (42) for `torch`, `numpy`, Python `random`, and HuggingFace `set_seed`. Choice-permutation RNG seeded separately at scoring time. All code at the public repo above.

4 Main Result

Our best leaderboard submission applies the v3 adapter ($r = 8$, all-7 projections, 384 px, 5 epochs) with smart-TTA at $K = 4$:

Configuration	**Public LB**
Random baseline (analytic)	~ 0.325
Pretrained, no fine-tune (200-ex val proxy)	~ 0.415
v1 fine-tuned, no TTA	0.76458
v1 fine-tuned + smart-TTA	0.77665
v2 (img 512) + smart-TTA	0.77867
v5 ($r = 4$, 5 epochs) + smart-TTA	0.79476
v4 (7 epochs) + smart-TTA	0.80281
v3 ($r = 8$, 5 epochs) + smart-TTA	**0.81287**
Top of leaderboard (for context)	0.89939

Table 3: Public leaderboard scores. Our best: v3 adapter + smart-TTA at 0.81287. The pretrained-baseline row is a 200-example val proxy and is not directly comparable to the LB rows (1,008-example hidden test); it is included to anchor the qualitative gap from no-fine-tune to fine-tuned.

Net lift from training-side improvements (v1 $\rightarrow$ v3, both LB): $+4.83$ pp. Net lift from the inference-only smart-TTA on v1 (also LB): $+1.21$ pp. The qualitative gap from the pretrained-baseline (~ 0.415 on val proxy) to fine-tuned (0.81287 LB) confirms that fine-tuning is the dominant source of accuracy, but the cross-split nature of that comparison precludes a precise pp-delta.

5 Ablations

The principal axes we vary: training duration, LoRA rank, image resolution, and TTA strategy. The "Train-val (4-bit)" column reports the val accuracy printed at the end of `Trainer.train()`; "LB" is the public leaderboard score under bf16 inference with smart-TTA.

A note on statistical power. All runs in this section use a single random seed; we did not have the GPU-quota budget to repeat each ablation across multiple seeds. With $n = 1\{,\}008$ test examples and accuracies in the 0.77–0.81 range, the binomial standard error of any single-run LB estimate is ≈ 1.3 pp. We therefore distinguish, in the analysis below, between deltas > 3 pp (interpreted as likely real) and deltas < 2 pp (interpreted as suggestive but indistinguishable from sampling noise without replication).

5.1 Headline Ablations Table

ID	Config	Train-val	LB
v1	$r = 8$, all-7, 384 px, 3 epochs (baseline)	0.7634	0.77665
v2	1. img_size=512	0.7586	0.77867
v3	**+ epochs=5 (best)**	**0.8015**	**0.81287**
v4	1. epochs=7	0.7996	0.80281
v5	$r = 4$, $\alpha = 8$, 5 epochs	0.7729	0.79476

Table 4: Five ablation rows. Train-val is 4-bit eval (training-time); LB uses bf16 + smart-TTA. v1 LB without TTA: 0.76458. The v3 row is bolded as our final submission.

5.2 TTA Strategy (No Retraining)

Measured on val with the v1 baseline adapter:

Strategy	Val acc	Δ vs no-TTA
No TTA ($K = 1$)	0.7290	—
Always-TTA ($K = 4$, all)	0.7405	+1.15 pp
Smart-TTA ($K = 4$, skip < 3)	**0.7481**	**+1.91 pp**

Table 5: TTA strategies on v1. Smart-TTA dominates; we use it for all submissions starting from v1+sTTA.

Per-bucket smart-TTA breakdown on v1 val ($n = 1048$): 2-choice 0.7992 ($n = 244$, identical to no-TTA, bypassed), 3-choice 0.7106 (+1.18 pp, $n = 508$), 4-choice 0.8492 (+2.78 pp, $n = 252$), 5-choice 0.3182 (+4.55 pp, $n = 44$). The pattern matches the pretrained-baseline analysis: position bias is largest where invalid letter slots are most numerous. For 5-choice questions where "E" is rarely seen at pretraining, the model's prior over letters is heavily skewed away from later positions; TTA's logit averaging cancels exactly that bias.

5.3 Image Resolution (v2)

The hypothesis was that ScienceQA's diagram-heavy questions would benefit from higher visual token resolution. Empirically, train-val *fell* by 0.48 pp (0.7634 → 0.7586) while LB *rose* by 0.20 pp (0.77665 → 0.77867); both deltas within one binomial standard error. With $n \approx 1000$ test examples and accuracy $p \approx 0.78$, SE $= \sqrt{\frac{p(1-p)}{n}} \approx$ 1.3 pp. We report this as **statistically tied with baseline**.

Three plausible explanations: (1) ScienceQA images are mostly < 512 px natively, so upsampling adds no information; (2) SmolVLM was pretrained at specific image scales, and 512-px tiles are mildly out-of-distribution; (3) at our 4.34M-parameter budget, visual fidelity is not the binding constraint — training duration is (see Section 5.5).

5.4 LoRA Rank (v5) and Decomposition

The capacity-halving control (v5: $r = 4$, $\alpha = 8$, 5 epochs, ~ 2.17M trainable params) yields **0.79476** LB — between v1 and v3. Crossing v1, v3, v5 isolates the two axes:

Comparison	Held fixed	Δ LB
$r = 4 \rightarrow r = 8$	epochs=5	+1.81 pp (v5→ v3)
3 → 5 epochs	$r = 8$	+3.62 pp (v1→ v3)

Table 6: Two-axis decomposition. Training duration is roughly 2 × the lever of rank.

Two findings: (1) **$r = 8$ was the right rank choice** — halving the rank costs 1.81 pp; the saved ~ 2.17M params cannot be redirected without violating the budget elsewhere. (2) **Training duration is roughly 2 × the lever of rank.** At our parameter budget and dataset size, fitting time is a stronger constraint than fitting capacity — consistent with the small-data regime (3,109 examples × 3 epochs ≈ 9,300 example-views).

5.5 Training Duration: The Convergence Curve

The single most impactful axis. Three measurements at epochs $\in \{3, 5, 7\}$ trace the generalization curve cleanly:

Epochs	Run	Train-val	LB
3	v1	0.7634	0.77665
5	**v3 (best)**	**0.8015**	**0.81287**
7	v4	0.7996	0.80281

Table 7: Convergence curve at $r = 8$. Both metrics peak at 5 epochs and decline at 7.

The diagnostic that motivated the v3 (3→ 5 epochs) ablation: v1′s training loss was *still descending* at step 580 (final step of epoch 3, loss 0.2535). The descent across 3 epochs — step 20 = 1.386 (warmup peak), 100 = 0.674, 200 = 0.519 (end of epoch 1), 400 = 0.323, 580 = 0.254 — showed continued progress with no clear plateau. We interpreted this as *under-training* rather than convergence: a 4.34M-parameter model on 9,300 example-views was plausibly insufficient. v3′s +3.62 pp LB lift confirmed this diagnosis.

The v4 result (5 → 7 epochs) is the textbook overfitting signature on small data, confirmed by both training-time and inference-time measurements (Table 7). Both curves peak at 5 epochs and decline at 7. Inter-curve agreement is itself evidence: a measurement artifact would show different patterns on train-val (which uses the val split) and LB (which uses held-out test), but here they agree directionally.

A subtler observation: train-val drops by only 0.19 pp while LB drops by 1.01 pp from v3 to v4. This *asymmetric* deterioration is consistent with classical overfitting — additional optimizer updates fit training-specific patterns that transfer partially to similarly-distributed val data but generalize less to held-out test. If the deterioration were noise alone, both metrics should drop by similar magnitudes (modulo each set's standard error of ≈ 1.3 pp).

This three-point curve provides empirical defense of v3 as near-optimal at this parameter budget and bounds the dominant-axis lever: training duration affects LB by ~ 5 pp across the explored range, larger than both rank (~ 1.8 pp) and visual resolution (~ 0 pp).

6 Discussion

6.1 The 4-bit-vs-bf16 Inference Gap

We trained with the base in 4-bit NF4 (QLoRA) and computed training-time val with the same 4-bit weights. The submission notebook runs offline and lacks `bitsandbytes`, so it loads the base in `bf16`. The LoRA adapter — optimized assuming 4-bit-dequantized weights — is then applied to the original `bf16` weights of the SmolVLM checkpoint, which differ from the dequantized 4-bit weights.

On v1 we measured this gap explicitly: training-time val (4-bit) = 0.7634, inference-time val (bf16, no TTA) = 0.7290 — a **3.44 pp drop** purely from precision mismatch, larger than the < 1 pp typically reported in the QLoRA paper. Our config (LoRA on all 7 projections, including high-magnitude MLP gate/up/down weights) is plausibly more sensitive to dequantization noise than attention-only LoRA, since MLP weights have larger magnitudes that benefit more from precise representation.

We attempted to evaluate inference at 4-bit but the offline-eval rule forecloses it without bundling `bitsandbytes` wheels into a Kaggle Dataset and pip-installing from local — engineering risk we judged disproportionate to the expected 1–3 pp lift. The lesson generalizes: **competition rules forbidding internet access at evaluation time effectively foreclose any inference-time technique that depends on non-default packages.**

6.2 Val-vs-Leaderboard Ranking Flips

The v1 → v2 comparison surfaced a striking val-vs-LB ranking inversion: v2 underperformed v1 on val (−0.48 pp) but slightly outperformed on LB (+0.20 pp). With sample sizes ~ 1,000 and accuracies in the 0.76–0.78 range, the standard error on each estimate is ≈ 1.3 pp, so both deltas are within one SE. We do not interpret this as v2 being genuinely better; we interpret it as evidence that **the two configurations are statistically tied** — which is itself useful, arguing against allocating capacity toward visual resolution and *for* allocating toward training duration. The v3 result then confirmed this hypothesis.

6.3 The Loss Curve as a Methodological Diagnostic

A common failure mode at small parameter budgets is to spend ablations on hyperparameters (LR, scheduler shape, optimizer details) rather than on whether the model has *seen enough data*. Our v1 loss curve — still descending at step 580 with no plateau — was the empirical signal that more training-data exposure would help. Reading this signal saved us from running half a dozen unproductive hyperparameter ablations. We recommend

this diagnostic step to anyone working at similar parameter-budget constraints: **before exploring the hyperparameter space, check whether the existing curve has converged.**

Combined with v4′s overfitting result, our ablation methodology was **data-curve-driven, not random hyperparameter search**. Each ablation tested a hypothesis: v3 tested "is the model under-trained?" (yes); v4 tested "does more help still?" (no, overfits); v5 tested "could we use half the params?" (no, costs 1.81 pp); v2 tested "does more visual capacity help?" (no, statistically tied).

7 Reproducibility

The code is public. The repository is organized so a reviewer can re-run end-to-end: `config.py` (CFG dataclass with all hyperparameters), `data.py` (dataset, prompt builder, loss-masked collator, Kaggle staging helper), `build_model.py` (QLoRA + LoRA wrapper, processor configuration), `scoring.py` (log-likelihood scoring + smart-TTA), `train.py` (HF Trainer entry point), `make_submission.py` (offline submission generator), `smoke_test.py` (end-to-end pipeline validation on tiny data), `p2p_train.ipynb` (Kaggle training notebook), `p2p_submit.ipynb` (Kaggle offline submission notebook).

Setup steps. (1) Fork the GitHub repo and link it to a Kaggle Dataset named `p2p-code`. (2) Upload SmolVLM-500M-Instruct weights to a Kaggle Dataset named `smolvlm-500m-instruct`. (3) Run `p2p_train.ipynb` with the competition data, `p2p-code`, and `smolvlm-500m-instruct` attached; set `CFG` overrides in cell 10 (e.g., `CFG.epochs=5`); click **Save & Run All**. (4) Publish the resulting adapter as a Kaggle Dataset. (5) Run `p2p_submit.ipynb` with internet OFF and the adapter dataset attached; output is `submission.csv` at `/kaggle/working/submission.csv`.

Documented landmines. The competition zip extracts with a duplicated `images/images/` directory, handled by `data.stage_competition_data()`. The `transformers v5` removal of `AutoModelForVision2Seq` is handled by a `try/except` import fallback to `AutoModelForImageTextToText`. CUDA state corruption from prior cell runs is handled by a defensive `empty_cache + synchronize` before `set_seed`. The QLoRA loss explosion is handled by the three fixes in Table 2.

8 Limitations and Future Work

Single-seed measurements; no error bars. Time and quota constraints prevented multi-seed runs. With $n \approx 1000$ and accuracies in the 0.78–0.81 range, single-run accuracy estimates have standard errors of ~ 1.3 pp; deltas < 2 pp should be interpreted as suggestive rather than significant.

No adapter ensembling. v1, v2, v3 are similar-architecture similar-data adapters whose errors are likely correlated, but a logit-average ensemble might still extract 0.3–0.7 pp; not implemented due to compute budget.

5-choice questions remain hard (0.32 with smart-TTA). The pretrained letter-distribution skew at "E" persists after fine-tuning on a small data slice ($n = 110$ in train). Targeted up-weighting or curriculum learning on choice-count buckets are unexplored levers.

The 4-bit→ bf16 gap is left on the table due to the offline-eval rule. If a future version of the rule permitted bundling `bitsandbytes` wheels (or the Kaggle base image included `bitsandbytes`), recovering the ~ 3 pp gap would likely vault the LB above 0.83.

The `solution` field in the dataset (gold reasoning chains) was not used. Including it as auxiliary supervision via chain-of-thought distillation could improve reasoning on multi-step questions; not tested.

Other ablations not run for this report: attention-only LoRA, MLP-only LoRA, and removing lecture from the prompt. Each would have added a row to Table 4; we judged the four axes already explored sufficient to defend the design.

9 Conclusion

Under the binding constraints of a fixed model (`SmolVLM-500M-Instruct`), a 5,000,000-trainable-parameter cap, and offline evaluation, we achieved **0.81287** on the public leaderboard via

QLoRA fine-tuning at rank 8 across all seven text-decoder linear projections with 5 epochs of training, augmented by smart choice-permutation TTA at inference. The most impactful single lever was extending training from 3 to 5 epochs (loss had not converged at 3); a further extension to 7 epochs reduced LB by 1.0 pp, establishing 5 epochs as near-optimal and bounding the convergence point empirically. Higher visual resolution (384 → 512 px) was statistically tied with baseline; halving the LoRA rank (8 → 4) cost 1.81 pp at matched training duration — together suggesting visual capacity and adapter rank were not the binding constraints at our parameter budget; *fitting time* on the small training set was. Three independent QLoRA-stability fixes (paged AdamW 8-bit, aggressive grad clipping, longer warmup) were essential for reproducible training on T4. The gap between training-time and inference-time precision (4-bit vs `bf16`) is forced by the offline-eval rule and accounts for ~ 3.4 pp of headroom that would otherwise be available; closing it would require pre-bundling quantization libraries, an engineering option we judged unfavorable.

AI tooling disclosure. Claude Code was used for code assistance, debugging, data analysis, and report drafting.